

EE619 Lab Manual

Section I: Initial Setup

1. The tools setup and the necessary files have been set up inside Docker. This can be thought of as a container, much simpler than a virtual machine.
To invoke the Docker image, first extract the tar archive by typing the following command in the terminal -

```
Shell
docker load -i ee619_docker_setup.tar
```

2. Now, invoke the Docker image using the commands

```
Shell
xhost +local:root
docker run -it --env="DISPLAY=$DISPLAY"
--volume="/tmp/.X11-unix:/tmp/.X11-unix:rw" ee619_lab:final bash
```

Important Note

Any changes you make inside the Docker container will be lost once you exit docker unlike in a virtual machine, where the changes get saved. You will need to commit the changes that you wish to save using the following commands. Open a separate terminal and type -

```
Shell
docker ps -a
```

This command will list all active containers. Locate the *Container ID* (alphanumeric code) for your container. For e.g. say the container ID is 22c8d3d6aae5 and your team name is Team1. Then type the following to save the changes.:

```
Shell
docker commit 22c8d3d6aae5 ee619_lab:Team1
```

To check the saved images, run the command

```
Shell
docker images
```

If you accidentally exit the Docker session or are beginning a new session, you can load the saved image by typing -

Shell

```
xhost +local:root  
docker run -it --env="DISPLAY=$DISPLAY"  
--volume="/tmp/.X11-unix:/tmp/.X11-unix:rw" ee619_lab:Team1 bash
```

Please ensure you save your container periodically.

3. Go to the home directory with the command

Shell

```
cd /home/user
```

Use the command **cd** whenever you want to go into a folder. Think of this as double-clicking the folder in Windows. If you want to see the contents of the current directory, type **ls** in the terminal.

4. You can view your current working directory with the command

Shell

```
pwd
```

5. The command below will display the *Icarus Verilog* version along with a list of options that can be used with the tool. Use this command to confirm Icarus Verilog is configured correctly.

Shell

```
iverilog -v
```

6. This command will display the Yosys version, confirming it is set up correctly.

Shell

```
yosys --version
```

7. This will display the OpenSTA version and a list of options that can be used with the tool

Shell

```
sta --version
```

Inform your TA that your tool setup is fine.

Section II: Counter design

Verifying the behavioral code

- Go to the location `/home/user/designs/counter` by typing -

Shell

```
cd /home/user/designs/counter
```

We will keep all design files associated with the counter in this directory.

- Observe the contents of this directory by typing `ls`. There will be two directories named *presynthesis* and *synthesis*. The initial behavioral level codes and testbenches will be kept in the *presynthesis* folder. All files related to synthesis can be kept in the */synthesis* folder. (An organization to make our lives easier).

Shell

```
cd presynthesis  
ls  
gedit counter.v &
```

This will open the *counter.v* file in a text editor. This describes a very simple counter. Observe the code.

You might see a lot of warnings in the terminal window when you open the file above. To get rid of them, use the command below instead.

Shell

```
gedit counter.v > /dev/null 2>&1 &  
gedit tb_counter.v > /dev/null 2>&1 &
```

This will open the testbench file. Observe the code.

- Run the simulation for this counter and verify the code using Icarus Verilog. (*source <filename>* reads and executes commands from a script file; this script file contains commands to run the Verilog simulation)

Shell

```
source sim_cmd  
gtkwave signals.vcd &
```

Show the results to the TA.

Observing a standard cell library

- We will be using the *Nangate 45nm* open source library for synthesizing our counter. The technology library file with Verilog description of the standard cells has already been copied into the Docker setup. Go to the following folder (this is a single command). ~ stands for home directory, which in this case is */home/user*.

Shell

```
cd  
~/NanGate_45nm_OCL_v2010_12/pdk_v1.3_v2010_12/NangateOpenCellLibrary_PDKv1_3_v2010_12/Front_End/Verilog
```

- Open the *NangateOpenCellLibrary_cells.v* file and observe the contents. DO NOT make any changes to this file.

Shell

```
gedit NangateOpenCellLibrary_cells.v > /dev/null 2>&1 &
```

- You can also open the library file with timing and power information about all these cells.

Shell

```
cd  
~/NanGate_45nm_OCL_v2010_12/pdk_v1.3_v2010_12/NangateOpenCellLibrary_PDKv1_3_v2010_12/Front_End/Liberty/CCS  
gedit NangateOpenCellLibrary_typical_ccs.lib > /dev/null 2>&1 &
```

Compare *AND2_X1* (2-input AND gate with minimum drive strength) in both files. Can you see how the delays and the power consumption have been characterized for the cell? How many *AND2* gates are there in the library? Feel free to Google if you don't understand a particular terminology in the .lib file.

Inform your TA that you have completed this step.

Synthesizing the counter

We need to implement the counter in terms of the standard cells available in the **_cells.v* file. For this, we will use the open-source tool Yosys.

- Ensure that you are in the *synthesis* folder.

Shell

```
cd ~/designs/counter/synthesis
```

- The commands for synthesizing the code are available in *synthesis_script.tcl*. Open this file using gedit.

Shell

```
gedit synthesis_script.tcl > /dev/null 2>&1 &
```

- Invoke Yosys using the following command -

Shell

```
yosys
```

- Run the commands in the *synthesis_script.tcl* (tcl is pronounced as tickle) file one by one. The tcl file has been commented to help you understand the stages better. If you need to understand a command better, you can execute **help <command>** in the yosys terminal. E.g., say you want to understand more about the *read_verilog* command. Run the following in the terminal. **help read_verilog**
- Observe the design at all stages using the **show** command. Can you spot the *AND2_X1* cell in the final design?
- Once synthesis is completed, exit from yosys.

Shell

```
exit
```

- Open the *counter_synth.v* file and observe the contents.

Shell

```
gedit counter_synth.v > /dev/null 2>&1 &
```

Inform your TA that you have completed this step.

Now we need to verify that this netlist is the counter we designed. But before that, we analyze the timing and power consumption of the design.

Timing analysis of the synthesized design

We will use the open-source tool *OpenSTA* to perform static timing analysis (STA) on the synthesized design, checking setup and hold timing, and also estimate its power consumption. This requires the synthesized netlist, standard cell library (timing models), and timing constraints as inputs.

- Go to the synthesis folder.
- The commands for timing analysis are provided in *sta_script.tcl*. Open and review this file. Also, examine *constraints.sdc*, which contains the timing constraints applied to the design.

Shell

```
gedit sta_script.tcl > /dev/null 2>&1 &  
gedit constraints.sdc > /dev/null 2>&1 &
```

You can add more constraints in the sdc file, such as *set_input_delay* and *set_load*, depending on your requirement.

- Invoke the OpenSTA tool using the following command

Shell

```
sta
```

- Source *sta_script.tcl* within the OpenSTA session.

Shell

```
source sta_script.tcl
```

- Now, look at the commented commands in *sta_script.tcl* that begin with **report_***. Execute them one by one in the terminal and analyse the generated reports.

Shell

```
report_checks -path_delay min_max -format full_clock_expanded  
-fields {input_pin slew capacitance} -digits 3
```

- You can also save the timing report to a file in the following way -

Shell

```
report_checks -path_delay min_max -digits 3 > slack.txt
```

- View the power report with the command -

Shell

```
report_power -digits 3
```

Observe the different components of power reported, such as leakage, switching, and internal power.

Show the results to your TA.

- Exit from OpenSTA

Shell

```
exit
```

Verifying the synthesized netlist

To verify the design, we need the synthesized netlist, the testbench and the Verilog description of the standard cells.

For this sample design, we will be demonstrating delay-annotated post-synthesis simulation, which requires an SDF (Standard Delay Format) file. This file contains delay information (cell and interconnect delays) extracted after STA and is used in gate-level simulation to model realistic timing behavior. (notice the **write_sdf** command at the end of *sta_script.tcl*, this generates the SDF file *counter_synth.sdf*)

- Go to the *post_synth_sim* folder.

Shell

```
cd ~/designs/counter/synthesis/post_synth_sim
```

- Now, here for simulation, we are using the testbench **tb_counter_with_sdf.v**
You may open this file using the command

Shell

```
gedit tb_counter_with_sdf.v &
```

Observe the **\$sdf_annotate** statement added: this reads delay information from the *counter_synth.sdf* file and annotates it onto the logic gates and flip-flops during simulation.

- Run the simulation using *Icarus Verilog* and visualise the waveforms in *GTKWave*

Shell

```
source sim_cmd  
gtkwave signals.vcd &
```

- Notice that the count value does not change exactly at the rising clock edge; there is a delay between the clock edge and the count update. This behavior reflects the delay-annotated simulation.

Do the waveforms match the waveforms we got before synthesis (with a propagation delay)? Congrats! The counter has been synthesized correctly.

- You can view the sdf annotation file as follows -
You are currently in the directory:
/home/user/designs/counter/synthesis/post_synth_sim
Type **pwd** to see the current directory. Then type

Shell

```
cd ..
```

(*..* refers to one level up from your current directory)

You will go to the directory: */home/user/designs/counter/synthesis*

Shell

```
gedit counter_synth.sdf &
```

Please observe the contents of this file.

Show the simulation results to your TA.

Tips

It can get cumbersome to type the commands in Yosys one by one. The *synthesis_script.tcl* can be sourced in one go while invoking yosys by typing the following -

Shell

```
yosys -s synthesis_script.tcl
```

If the *show* command is causing issues, simply comment out or remove those and source the script again. For example, the *synthesis_script_noshow.tcl* is a copy of the same script without the *show* command. You will be able to run this in one go as follows -

Shell

```
yosys -s synthesis_script_noshow.tcl
```

Section III: Synthesizing the backend (Project I)

- Download the Verilog files to this PC from mail using a web browser.
- Now we need to push these files into the Docker container. For this, a directory structure similar to the *counter* has already been created. Navigate to it using -

Shell

```
cd ~/designs/project1
```

- The simulation scripts have already been copied into the respective locations. Make sure to substitute the names of the Verilog files in the scripts with your code files, wherever needed.
- Identify the Docker container ID by running the following command in a new terminal window -

Shell

```
docker ps -a
```

- Say the container ID is *22c8d3d6aae5*. Assume the Verilog file to be copied is *backend.v*. The following command will copy it into */home/user/designs/project1/presynthesis* - (single command)

Shell

```
docker cp <path_to_Verilog_file>/backend.v  
22c8d3d6aae5:/home/user/designs/project1/presynthesis/
```

- Confirm that the files were successfully copied into the Docker container.
Inform your TA that you were able to copy the files successfully into the Docker container.
- Now you can proceed with synthesizing your backend code. You may have to edit your code to make it synthesizable. Please make sure you save your code and commit the changes in the Docker container periodically to avoid losing progress. Do your best!
- While running STA, you need to change the constraints.sdc file as follows to declare the i_clk and i_sclk as different clock domains:

```
# Define 50 MHz clock (20 ns period) with 50% duty cycle on input port 'clk'  
create_clock -name CLK_MAIN -period 20 -waveform {0 10} [get_ports i_clk]  
create_clock -name CLK_SCLK -period 20 -waveform {0 10} [get_ports i_sclk]
```

```
# Asynchronous domains  
set_clock_groups -asynchronous \  
-group [get_clocks CLK_MAIN] \  
-group [get_clocks CLK_SCLK]
```

Show the simulation results to your TA once you have the synthesized code working.